

SOFTWARE ENGINEERING PROCESS MODELS

Agile Development Methodologies

-Dr Madhavi Dave



Software Engineering | Process & Project Management Models

What Is Agile Development?

Agile software development is an **iterative and incremental** approach to building software. Instead of delivering the entire product at the end of one long cycle, the work is broken into short, repeatable iterations that each produce a small, usable piece of working software.

The core idea: plan a little, build a little, get feedback, and adapt — continuously — rather than trying to freeze requirements upfront and follow a rigid plan for months.

Origins: In February 2001, seventeen software practitioners met in Snowbird, Utah, frustrated with heavyweight, documentation-driven processes like Waterfall. They published the **Agile Manifesto**, which now underpins every modern Agile framework.

PLAN → BUILD → REVIEW → ADAPT



The cycle repeats every 1–4 weeks throughout the project, continuously re-prioritizing based on real feedback.

Four Core Values

"We are uncovering better ways of developing software by doing it and helping others do it."

Individuals & Interactions



over processes and tools

Working Software



over comprehensive documentation

Customer Collaboration



over contract negotiation

Responding to Change



over following a plan

Note: items on the right still have value — Agile simply values the items on the left more.

Twelve Guiding Principles

1

Satisfy the customer through early and continuous delivery of valuable software.

2

Welcome changing requirements, even late in development.

3

Deliver working software frequently — in weeks rather than months.

4

Business people and developers must work together daily.

5

Build projects around motivated individuals; trust them to get the job done.

6

Face-to-face conversation is the most efficient way to convey information.

7

Working software is the primary measure of progress.

8

Agile processes promote sustainable development at a constant pace.

9

Continuous attention to technical excellence and good design enhances agility.

10

Simplicity — maximizing work not done — is essential.

11

The best architectures and designs emerge from self-organizing teams.

12

Regularly reflect on how to become more effective, then adjust.

Traditional (Waterfall) vs. Agile Development

Aspect	Waterfall (Traditional)	Agile
Approach	Sequential — one phase completes before the next begins	Iterative & incremental — cycles of a few weeks
Requirements	Fixed and frozen upfront	Expected to evolve; refined continuously
Delivery	A single delivery at the very end	Frequent, working increments throughout
Customer Involvement	Mainly at the start and the end	Continuous, throughout the project
Response to Change	Difficult and costly once underway	Welcomed at any stage
Testing	A distinct phase after development	Continuous, alongside development

Key Characteristics of Agile Development

1

Iterative & Incremental

Work is delivered in short cycles, each producing a usable slice of the product.

2

Cross-Functional Teams

Small, self-organizing teams own analysis, design, coding, and testing together.

3

Continuous Feedback

Customers and stakeholders review progress every cycle, not just at the end.

4

Adaptive Planning

Plans are revisited and re-prioritized every iteration instead of frozen upfront.

5

Working Software First

Progress is measured by what runs and passes tests, not by documents produced.

6

Continuous Improvement

Regular retrospectives turn lessons learned into changes for the very next cycle.

Major Agile Methodologies at a Glance

1

Scrum

Fixed-length sprints, defined roles (PO/SM/Team), and a cadence of planning, daily standups, review & retrospective.

2

Extreme Programming (XP)

Engineering-focused: pair programming, TDD, continuous integration, and frequent small releases.

3

Kanban

Visualizes work on a board and limits work-in-progress to optimize continuous flow, without fixed iterations.

4

Lean Software Development

Adapts Lean manufacturing principles: eliminate waste, amplify learning, deliver fast, empower the team.

5

Crystal

A family of methods tuned by team size and project criticality; prioritizes people over process.

6

Feature-Driven Development

Organizes work around building and designing small, client-valued features in short iterations.

7

DSDM

Business-driven framework that fixes time and cost, then flexes scope to meet the deadline.

8

Adaptive Software Development

Replaces fixed planning with a Speculate–Collaborate–Learn cycle for high-uncertainty projects.

Scrum Roles



Product Owner

Owns and prioritizes the Product Backlog; represents the customer and business stakeholders; decides what gets built and in what order.



Scrum Master

Facilitates Scrum events, removes impediments blocking the team, and coaches everyone in Agile practice — a servant-leader, not a manager.



Development Team

A small, cross-functional, self-organizing group that designs, builds, and tests the increment every sprint.

Scrum Events (Ceremonies)

Sprint Planning

The team selects backlog items for the upcoming sprint and defines a sprint goal.

Start of sprint

Daily Scrum

A 15-minute standup where the team syncs on progress and flags blockers.

Every day

Sprint Review

The team demonstrates the working increment to stakeholders and gathers feedback.

End of sprint

Sprint Retrospective

The team reflects on how it worked and agrees on improvements for next time.

End of sprint

THE SPRINT

All four events happen inside a Sprint — a fixed-length container, typically 2 to 4 weeks, that repeats for the life of the project. The sprint itself never changes length once started.

Scrum Artifacts



Product Backlog

An ordered, ever-evolving list of everything that might be needed in the product — features, fixes, and improvements — owned and prioritized by the Product Owner.



Sprint Backlog

The subset of Product Backlog items selected for the current sprint, plus the team's plan for delivering them and the sprint goal.



Increment

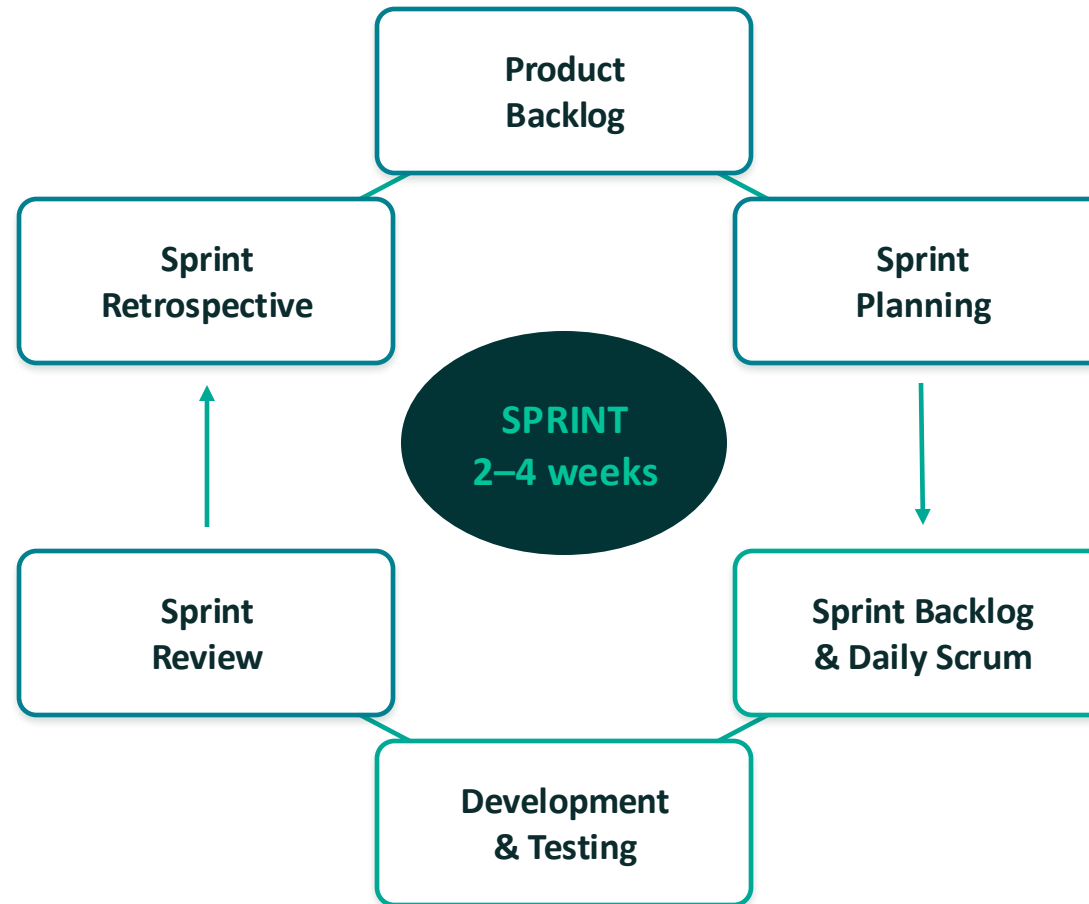
The sum of all completed backlog items at the end of a sprint — a working, potentially shippable piece of the product that meets the Definition of Done.

DEFINITION OF DONE

A shared checklist (e.g. coded, tested, reviewed, documented, integrated) that a backlog item must satisfy before it can count toward the Increment. It keeps “done” consistent across the whole team.

The Scrum Sprint Cycle

A hexagonal view of how the backlog, events, and increment connect into one repeating cycle.



The cycle restarts at Product Backlog every sprint — items completed become part of the shippable Increment; items not finished return to the backlog for reprioritization.

Extreme Programming (XP) — Key Practices

1

Pair Programming

Two developers work at one workstation — one writes code, the other reviews it in real time.

2

Test-Driven Development

Write an automated test before writing the code that makes it pass.

3

Continuous Integration

Merge and build code many times a day to catch integration issues immediately.

4

Refactoring

Continuously improve the internal structure of code without changing its behavior.

5

Small Releases

Release small, working versions of the system frequently.

6

Collective Code Ownership

Any team member can improve any part of the codebase at any time.

7

Simple Design

Build the simplest design that satisfies today's requirements — nothing more.

8

Coding Standards

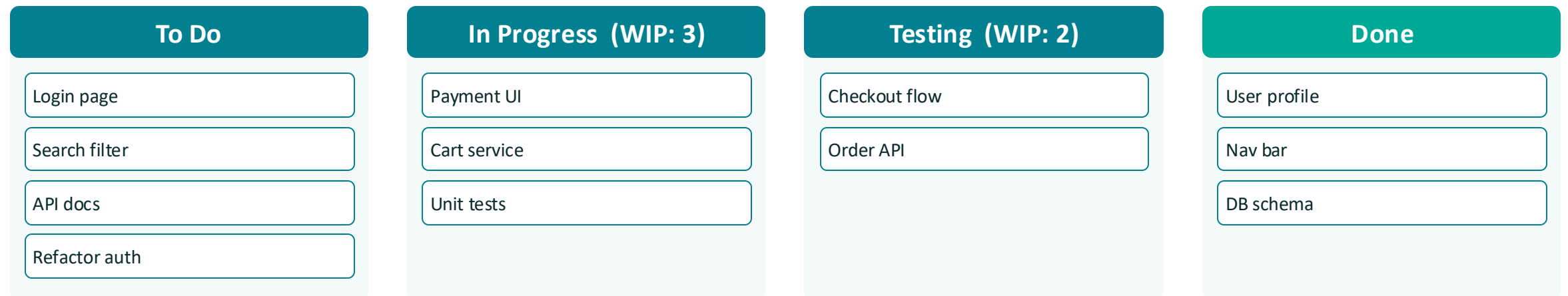
The whole team codes to one shared, consistent style.

Kanban — Visualize & Limit Work in Progress

Core Principles

- Visualize the workflow on a shared board.
- Limit Work in Progress (WIP) at each stage.
- Manage and measure the flow of work.
- Make process policies explicit.
- Improve collaboratively, using metrics and feedback.

Unlike Scrum, Kanban has no fixed-length iterations — work items move continuously as capacity allows.



Lean Software Development & Other Frameworks

Lean Software Development — Seven Principles

Eliminate waste

Amplify learning

Decide as late as possible

Deliver as fast as possible

Empower the team

Build integrity in

See the whole

Crystal

A family of methodologies (Clear, Yellow, Orange...) tuned by team size and project criticality; prioritizes people and communication over rigid process.

Feature-Driven Development (FDD)

Organizes work around designing and building small, client-valued features through short, repeatable design-then-build iterations.

DSDM (Dynamic Systems Dev. Method)

A business-driven framework that fixes time, cost, and quality upfront, then flexes the feature scope to hit the deadline.

Adaptive Software Development (ASD)

Replaces fixed, sequential planning with a Speculate → Collaborate → Learn cycle suited to high-uncertainty projects.

Benefits of Agile Development

1

Faster Time to Market

Frequent releases mean valuable features reach users sooner, not all at once at the end.

2

Higher Customer Satisfaction

Continuous collaboration ensures the product stays aligned with what customers actually need.

3

Better Adaptability

Changing requirements are absorbed each iteration instead of derailing the whole plan.

4

Improved Quality

Continuous testing and integration catch defects early, when they're cheapest to fix.

5

Stronger Team Morale

Self-organizing teams with clear ownership tend to be more engaged and motivated.

6

Reduced Project Risk

Small, frequent deliveries surface problems early rather than at a single, high-stakes finish line.

Challenges & Limitations of Agile

1

Requires Cultural Change

Moving to Agile often means restructuring teams, roles, and how success is measured.

3

Needs Disciplined, Experienced Teams

Self-organization works best with skilled, communicative teams — it can flounder without them.

5

Documentation Can Suffer

Minimal documentation can be a real problem in regulated or safety-critical industries.

2

Hard to Fix Cost & Timeline Upfront

Evolving scope makes it difficult to commit to a fixed price or date at project start.

4

Difficult to Scale

Coordinating many Agile teams on one large, distributed product requires extra frameworks (e.g. SAFe, LeSS).

6

Risk of Scope Creep

Openness to change can be misused to keep adding work without re-evaluating priorities.

When Should You Choose Agile?

Good Fit for Agile

- Requirements are unclear, evolving, or likely to change.
- Fast, incremental delivery of value is a priority.
- The customer can stay closely engaged throughout.
- Team size is small to medium and co-located or well-connected.
- The project is innovative or exploratory in nature.

Less Suited for Agile

- The contract fixes scope, price, and date all at once.
- The system is safety-critical and needs heavy upfront documentation.
- The team is large and widely distributed without strong tooling.
- Stakeholders cannot commit to ongoing engagement.
- Requirements are genuinely stable and well understood in advance.

SUMMARY

Key Takeaways

1 Agile is an iterative, incremental approach built on the 2001 Agile Manifesto's four values and twelve principles.

2 It replaces Waterfall's single, big-bang delivery with frequent working increments and continuous customer feedback.

3 Scrum (roles, events, artifacts), XP, Kanban, and Lean are the most widely used frameworks — each emphasizing a different facet of agility.

4 Agile delivers faster feedback, better adaptability, and higher quality — but demands cultural change, discipline, and engaged stakeholders.

5 The right choice depends on context: Agile suits evolving, collaborative projects; fixed-scope or heavily regulated work may still favor a plan-driven approach.